SQL Essentials and Normalization

Database Management Systems - Units 2-3 - Querying and schema design

Core SQL statements

```
SELECT dept, COUNT(*) AS staff, AVG(salary) AS avg_pay
FROM   Employee
WHERE  active = 1
GROUP  BY dept
HAVING COUNT(*) >= 3
ORDER  BY avg_pay DESC;
```

WHERE filters rows before grouping; HAVING filters groups after aggregation. The logical order of evaluation is FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

Joins

- INNER JOIN returns only matching rows from both tables.
- LEFT JOIN keeps every row from the left table and fills missing right-side columns with NULL.
- FULL OUTER JOIN keeps unmatched rows from both sides.
- A subquery can replace many joins, but a join is often easier for the optimizer.

Functional dependencies

A functional dependency X -> Y means that whenever two rows agree on X they must agree on Y. For example roll_no -> name. Normalization uses dependencies to remove update, insert and delete anomalies.

Normal forms

- 1NF: every attribute is atomic - no repeating groups or lists in a cell.
- 2NF: 1NF plus no partial dependency (a non-key attribute depending on only part of a composite key).
- 3NF: 2NF plus no transitive dependency (non-key -> non-key).
- BCNF: for every non-trivial dependency X -> Y, X must be a super key.

> *Example: Order(order_id, product_id, product_name, qty) with product_id -> product_name violates 2NF. Split into Order(order_id, product_id, qty) and Product(product_id, product_name).*

Transactions and ACID

- Atomicity: all operations of a transaction happen or none do.
- Consistency: a transaction moves the database from one valid state to another.
- Isolation: concurrent transactions do not see each other's partial work.

- Durability: committed changes survive crashes.

Isolation levels (read uncommitted, read committed, repeatable read, serializable) trade correctness against concurrency by allowing or preventing dirty reads, non-repeatable reads and phantom rows.